

Piloting Lean-Agile Hardware Development

Maarit Laanti
Nitor Delta
Kalevankatu 3 A 45
00100 Helsinki, Finland
Maarit.Laanti@nitor.fi

ABSTRACT

Lean-Agile hardware development is not only possible, but brings multiple benefits. The key to apply Lean and Agile thinking in hardware development is to understand the natural development iterations, and utilize the received feedback for the maturization of the hardware. The business owners need to sit on the driver's seat, as it is crucial to understand what business benefit can be received by making the development incremental. The benefits of incremental and iterative development of hardware may be as high as with software development. This paper describes a new kind of Lean-Agile Framework for hardware development; its background and experiences of its usage.

CCS Concepts

• **Software and its engineering** → **Software creation and management** → **Software development process management** → **Agile software development** • **Computer systems organization** → **Embedded and cyber-physical systems** → **Embedded hardware**.

Keywords

Large-scale agile software development, agile methods, hardware engineering, software engineering, software development, Scaled Agile, embedded systems, electrical engineering, consumer electronics, large scale agile.

1. INTRODUCTION

More and more companies are looking into ways to apply agile learning in electrical engineering, i.e. hardware development. One example of this is Wikispeed, an agile community project building low-cost and fuel-efficient car while reaching a five-star collision test rating. [1]. Advancing agile methods into product development has led also to publication of the Scaled Agile Framework 4.0 that supports hardware development [2] with a case study [3].

The success of any embedded product development project is highly dependent on hardware maturization, how the design evolves by the feedback received on the design created and tested.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Copyright held by Owner.

XP '16 Workshops May 24-24, 2016, Edinburgh, Scotland Uk
ACM 978-1-4503-4134-9/16/05.

<http://dx.doi.org/10.1145/2962695.2962698>



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

Harley-Davidson has found that the time that it takes for a new design to go through these iteration cycles is critical for the success of the new product development projects [4].

Agile hardware development or co-design of software and hardware together might also lead to multiple benefits, including reduced time and cost [5].

This paper describes an attempt to pilot a set of selected practices from software development on multiple hardware development projects in a Fortune 500 consumer electronics company in 2010-2011. There were several hundreds of developers involved, and multiple different hardware engineering areas, or disciplines and multiple projects with various front-end suppliers.

1.1. Background

The idea of piloting selected lean-agile practices emerged after the software organization was already fluent with using scaled agile methods in their development [6]. The software project and available hardware prototypes were on the critical path. The hardware prototype version launches were critical events when planning the Features to the next software platform increment. The success of the software project was highly dependent on these launches.

The traditional assumption in the organization was that the hardware (waterfall) projects are predictable, but software projects are causing the delays. Now, with the software increment planning the impact of delayed hardware was visible for the first time. We were able to estimate the impact of the hardware delay to the overall and software schedule, which was a multiplier of the delay detected on the hardware side. We were also able to show which software Features we had to postpone and why (i.e. what was the impact of the delay) and ask for management to keep the original deadline or drop some of the features.

2. WORKSHOPS

For half a year, we ran multiple workshops with a group of senior managers who had not just background in hardware development who knew the processes of their own area well. The workgroup consisted of roughly 10 experts representing different areas: printed board design, camera and display technologies, just to mention a few.

We started by discussing what was wrong in the current way of managing projects. The list was extensive: project plans were based on former experience, not on low-level estimates; a lot of emphasis was put into reporting but the reporting was only about the gap between the estimated current state and the wishful plan; when

projects were delayed, more people and more milestones were added; milestone approvals took a long time and yet all the milestones were eventually approved with exceptions. Adding more milestones and people to already delayed project are typical traditional corrections that Reinertsen states hurt more than help because the actions increase control thus slowing the development flow [7].

The existing hardware development processes assumed stable requirements and that planning for the entire design could be done at once – but in reality the requirements were changing during the project, and there was changes in technology, customer needs and markets.

The calendar time between milestones was between 3 months and up to more than a year and each hardware milestone consisted of up to 13 sub-reviews cascaded up hierarchically. Also the existing way of designing hardware did not go well hand-in-hand with agile SW development.

To our surprise most of the problems created by the waterfall process were exactly the same as on the software side – caused by the traditional approach to projects itself. However, due to the different nature of software and hardware development the solutions may not be exactly the same.

We had several discussions on the applicability of the lean and agile approaches to hardware development, and finally selected a few practices for a pilot. In this paper I will focus on the experiences from just four practices: 1) two-level rolling planning, 2) cadence, 3) synchronization, and 4) key decision points. The other practices tried out were flexible product development as defined by Preston Smith [8] and visionary concepting, where the ideas were taken from the car industry developing concept cars.

The four practices described in this paper were selected because the participants felt that this would change the traditional project management into Lean and Agile. At this point the workgroup participants confessed they had been thinking there was no way Lean Thinking and Agile Development could be utilized in hardware development but they were convinced by these series of workshops together that it was not just possible but also very beneficial!

3. FINDING THE SOLUTION

Scrum is partially based on Takeuchi's and Nonaka's paper "The New New Product Development Game" [9]. In this paper Takeuchi and Nonaka study companies that develop new products, and conclude that a game-like setting could be more beneficial for product development than phase-gate development, see Figure 1. One of the companies studied in this paper is Honda. However, Honda is not mainly a software company, but identifies itself as an engine company. Thus Schwaber and Sutherland, when inventing Scrum, actually took some of the Lean practices and started to use those in software development! Yet mechanical engineering is not that different from electrical engineering – so there really is no reason why the same principles could not be utilized in electrical engineering as well. Is the reason why we started to apply Lean Principles from Honda to software development because of the urgency to find some better working methods than those that existed at the time?

One of the differences between hardware and software development is that you can split the software into (almost) arbitrary sized bites but not the hardware iterations. For example

according to the Scaled Agile Framework guidelines one can fix the planning dates almost to any possible date [10]. With hardware this would make no sense – you only get the feedback at the end of iteration. Having a planning event in the middle of real hardware iteration would only cause turmoil – one would need to change the plans anyway when the actual real feedback becomes available. Imagine a situation where you have just planned the next increment a couple of days before receiving a new prototype build that you can then test only to find a critical error which makes your previous increment plan instantly invalid.

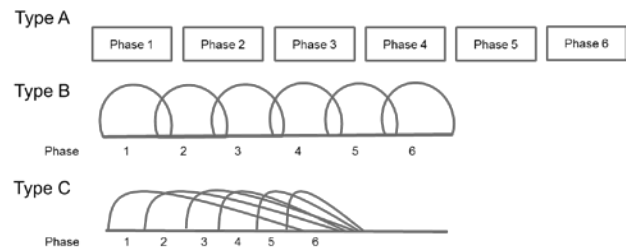


Figure 1. Different project management styles according to Nonaka and Takeuchi [9]

Yet the need for incorporating changes (or redefine the design based on the most recent learned information) is the same for both software and electrical engineering. Ken Rubin has [11] noted that the cost-of-the-change curve of hardware development rises more rapidly than the cost-of-the-change curve in software development, but one can influence it by choosing the tools and techniques in such a way that changes are allowed in the areas that can benefit from the delayed decisions [11].

Oosterwal [4] states also that understanding the iteration lengths is essential in order to learn how to control the new development projects. Project data from milestones from the past five years showed no correlation at all with the project success. However, when they started to understand the iteration lengths of different parts at Harley-Davidson, they found something that correlated with the project success. A project success was dependent on if a part could iterate and find a final mature design in the available time.

We concluded that we should understand too how long our iterations were and how many of these iterations we typically needed in different types of projects so that we could also create such a development frame that would predict the success and that could replace traditional phase-gate models.

4. THE NEW LEAN-AGILE DEVELOPMENT FRAMEWORK

We interviewed people who had been working in hardware development for a long time. We had a long discussion if each of the iteration loops starts when the new hardware design is sent to the printed circuit board manufacturer or when we receive these prototypes i.e. builds from the vendor. We then concluded to base the iterations on Build Maturity Reviews (BMRs) that took place when the new prototype could be tested and measured. At a BMR the hardware designer would know how many errors there were and how much rework the design may need. We also concluded that

whenever a completely new hardware design is made, we need at least three iterations to reach maturity. This was based on the former experiences of similar designs.

One area that caused a lot of discussion was how we could replace the milestones with a more Lean and Agile way. In software development we had abandoned milestones, and we were basing the progress on the actual measured developed functionality (delivered Features) during the development increments. We agreed the current decision-making was too heavy – yet there were very important and expensive decisions to be made so some formal approvals were needed.

One reason why the former hardware milestone reviews were so heavy was that those milestones combined both the content review and the decision-making. Our approach was to separate these too. The purpose was to check the quality at the lowest possible level. We aimed for high-quality feedback given by a colleague who had done similar work, not by a high-level manager who no longer knew about the latest technologies. Thus the cadence meetings were born.

Figure 2 presents the summary of the piloted practices.

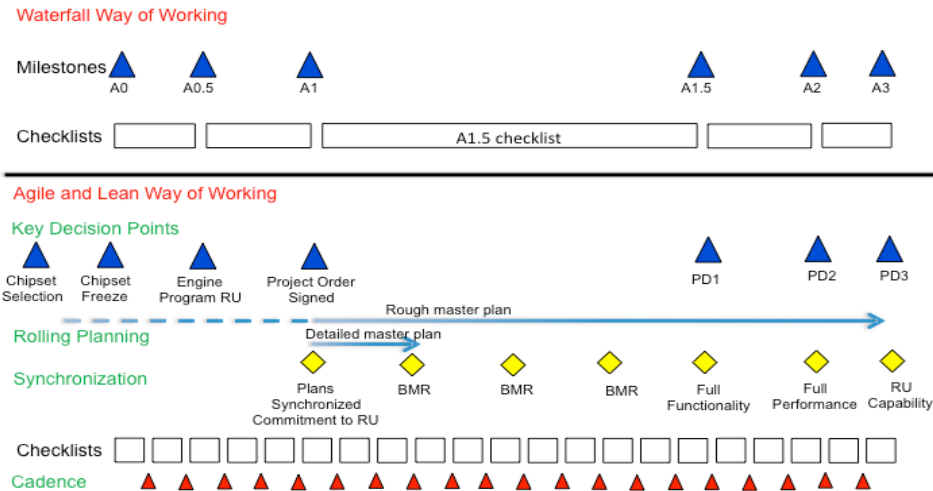


Figure 2. Agile and Lean way-of-working compared to Waterfall

4.1. Cadence

Cadence meetings were defined as:

- Cadence is not about reporting – it is about reviewing!
- Regular and predictable reviews on the team level
- Held every 6 weeks
- Peer-to-peer review (e.g. teams from all programs are present in the same cadence review) in order to have independent reviewers
- Facilitated by the line organization
- Different team inputs are reviewed by the other teams
- The team should be represented by the needed team members (project leader is mandatory)
- Output from the cadence will be input to the Synchronization and Key Decision Points
- Separates technical reviews (Cadence) from business decisions (Key Decision Points)
- Reduces risk since the work is reviewed frequently by independent reviewers
- Allows to introduce learning cycles – each cadence becomes a learning cycle
- Will enable more fluent resource pooling since all programs are reviewing each others' plan and development on a monthly basis, and it will be easier to move resources between programs when required

The principle in Cadence meetings was to review whatever was ready at the time against a checklist. As we had no other basis for checklists, we took the previous milestone quality criteria as the

basis of these peer reviews. See Figure 3 for a more detailed explanation of Cadence meetings.

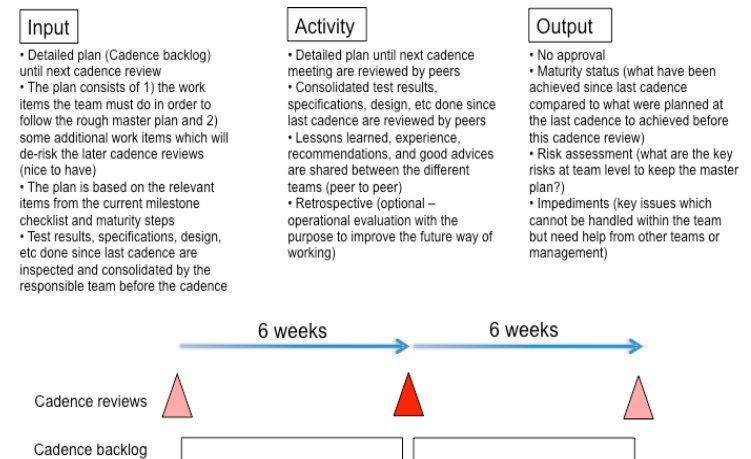


Figure 3. Cadence meetings explained

4.2. Synchronization

The synchronization points were defined as:

- A cross-functional integration point where we synchronize maturity, plans, risks, and impediments
- Synchronization means meetings with participants from all cross-functional areas including suppliers, product programs, and relevant teams

- Will not happen at regular intervals but will take place when it makes the most sense from an execution and delivery point of view
- Synchronization is a program-specific meeting (not mandatory to have participants from other programs present)
- Output from the Synchronization will be input for the Key Decision Points
- No business decisions are taking place in these meetings (this is done at the Key Decision Points)
- It is recommended that planning & retrospectives are carried out as part of Synchronization meeting
- Synchronization meeting is not very different from traditional project planning sessions, but happens more often and regularly
- Rolling planning is taking place between the synchronization points

Synchronization points are BMRs with full functionality and full performance tested and measured.

4.3. Two-level rolling planning

Two-level planning is a Lean practice. The idea is to keep the overall (or long-term) schedule on a rough level and the short-term schedule on more accurate level. When a term (or increment) is completed, the rough schedule is updated by the learning.

Since delivering only half-baked hardware makes no sense, we decided to draft the whole schedule for the whole project – but follow the Lean principle of keeping the long-term schedule on a rough level. We defined accurately only the next short term, i.e. next hardware iteration, which was the design until the next Build Maturity Review.

The rolling planning was defined as follows:

- Rolling planning type used is a two-level rolling planning. The master plan for the entire program is planned roughly until the end of the program and detailed until the next Synchronization Point (or as long as required)
- There is also ability to adapt to changes happening during the course of the program as we plan to re-plan
- As progress is made, more information will be gathered and thus impacting next round of the detailed planning
- Is taking place at the Synchronization Points (meetings).
- Joint and cross-functional planning guarantees commitment. Short-term targets and goals increase motivation.
- Dependencies are managed through joint planning

See Figure 4 for the description of Synchronization and rolling planning.

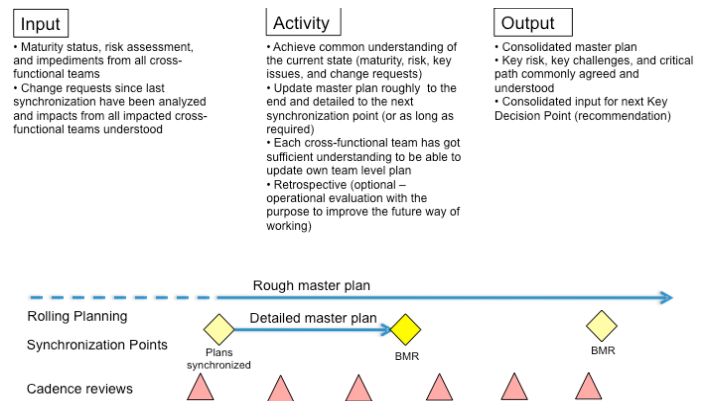


Figure 4. Synchronization and rolling planning

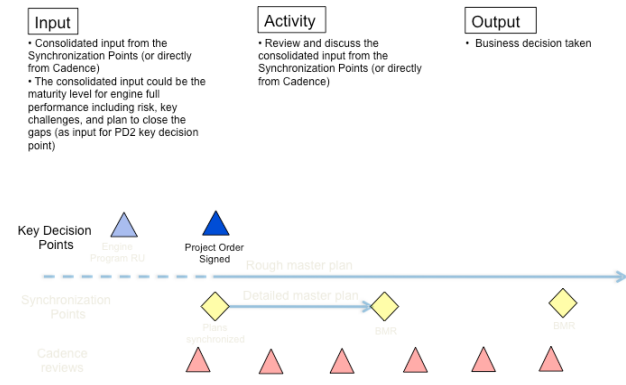


Figure 5. Key Decision Points

4.4. Key Decision points

The Key Decision points were defined as:

- Business decisions that happen when investment decisions are required
- Are *not* technical reviews
- Synchronization & Cadence provide input to Key Decision Points
- Are *not* gating the technical execution

Key Decision points were major events like engine program ramp up, project order signed, bigger milestones, etc.

See Figure 5 for Key Decision points explanation.

5. REFLECTIONS

This pilot was the first electrical engineering program / hardware project ever done in record time and with high quality in this Fortune 500 company. Oddly though, this result was not communicated nor celebrated at all.

The Cadence reviews were causing the most critique. Hardware designers just hated to show their work to their peers. The resistance was so huge that the Program Manager concluded that this experiment was not a good one because of the resistance he encountered. He also stated he should have had more support for

the change – he had to take all that critique by himself, he should have needed at least one coach per team to help with the change.

At that time we were not aware of the Braintrust meetings held at Pixar. Nothing could be more crucial to success of a new project than candor and critique [12]. Yet you need to have the right kind of atmosphere with no formal authority for these meetings to work properly [13]. Perhaps we were missing the right kind of atmosphere.

Rolling planning was the practice that got the most praise. It was also the easiest change to make as it involved only for a small number of people (project and program managers). Unfortunately we failed to implement the incremental design in full scale.

Table 1. Making hardware and HW drivers incrementally.¹

Function	1 st Build	2 nd Build	3 rd Build	Ramp-Up
HW	Initial (TS) Samples – limited functionality	First Engineering (ES) Samples – full functionality, possible errors	Later Engineering (ES) Samples – full functionality, fixed errors	Fully verified*, ready-for-mass-production (CS) samples
HW Driver	Production Test functionality (to prove physical connections only)	Basic Functionality in mandatory areas	Full functionality in mandatory areas; possible additional functionality	Errors fixed, ready for ramp-up
Feature				

In general the idea of incremental design was causing problems for us. People felt the hardware should have been created and tested as one entity. There are also reasons why the lowest-level software (hardware drivers) should rather follow the hardware schedules, than the Features in Program Increments. This is due to the tight coupling between hardware drivers and the hardware itself:

- Driver software may need to wait for hardware development
- Environment / platform (needed for testing)
- Cycle of tests should align to new HW module version availability

The former belief was that the entire hardware with the fully developed software drivers would be needed in order to fully test the hardware before the product could be released. If we had had a more incremental mindset, we could have not just developed but also delivered the hardware in increments, i.e. by delivering some functionality first and the additional features or functionality at a later release. Table 1 gives an idea how incremental development could be combined with the software development while aiming at one single release.

The challenge with incremental product development though is that many development errors may be results of interactions (and dependencies) within the integrated product. Those won't be known / can't be resolved until all software is completed. Any means to add modularity and increase functionality that is done by software (instead of hardware) would help.

6. CONCLUSIONS

Based on this experience, the best practices when doing Lean and Agile hardware development include:

- Start from strategic planning and see how you could benefit from iterative and incremental thinking in your product, e.g. planning multiple solutions for technically challenging problem areas (to diminish the technical risk)
- Plan the rough schedule to cover the whole product and all increments; plan only the next increment accurately. Allow changes to happen as needed.
- Hardware is not flexible the same way software is. However iterations can be used to mature the design. Figure out the natural iterations and plan the increments using those boundaries. Monitor the errors in the longest iterations in order to keep the project on track and to meet the expected market window.
- Use peer feedback to improve the product and the quality, but make sure to keep the right kind of atmosphere.

Special thanks to all former colleagues, and especially to Søren Karlsberg and Chris Dawson, for the ideas incorporated in this paper.

7. REFERENCES

1. Wikispeed, <http://wikispeed.org>, accessed 30.12.2015
2. SAFe 4.0, <http://www.scaledagileframework.com>, accessed 30.05.2016
3. SK Hynix Case Study, <http://www.scaledagileframework.com/hynix-case-study/>, accessed 30.5.2016
4. Oosterwal D, The Lean Machine: How Harley-Davidson Drove Top-Line Growth and Profitability with Revolutionary Lean Product Development. AMACOM, 2010. ISBN-13: 978-0814432884.
5. Punkka T, Agile Hardware and Co-Design. Embedded Systems Conference 2012, Boston, ESC-3008.
6. Laanti M, Agile Methods in Large-Scale Software Development Organizations. Applicability and model for adoption. Dissertation. University of Oulu, 2013. ISBN 978-952-62-0033-0
7. Reinertsen D, The Principles of Product Development Flow. Second Generation Lean Product Development. Celeritas Publishing, 2009.
8. Smith P, Flexible Product Development Building Agility for Changing Markets. JosseyBass, 2007. ASIN: B00DU7OJPW.
9. Takeuchi H and Nonaka I, The new new product development game. Harvard Business Review 64:1:137-146 (Jan/Feb) 1986.
10. Rubin K, blog on August 18th 2015, Agile in a Hardware / Firmware Environment – Draw the Cost of Change Curve <http://www.innolution.com/blog/agile-in-a-hardware-firmware-environment-draw-the-cost-of-change-curve> Accessed 30.12.2015.

¹ The table above shows the 'typical' expectations for Program builds. There is likely to be variation from Program-to-Program, specific area of development and sample maturity at the start of the Program lifecycle.

* All HW would be needed to be ready for full HW verification tests, including environmental tests and proven capability for mass manufacturing.

11. Leffingwell D, Agile Software Requirements. Lean Requirements Practices for Teams, Programs, and the Enterprise. Addison-Wesley, 2011. ISBN-10: 0-321-63584-1, ISBN-13: 978-0-321-63584-6.
12. Catmull E, Creativity, Inc.: Overcoming the Unseen Forces That Stand in the Way of True Inspiration. Random House, 2014. ISBN-13: 978-0812993011.
13. Catmull E, Inside the braintrust. Video, <https://www.youtube.com/watch?v=I1Mr3oKR7oM>, accessed 30.12.2015.